

# OPERATING SYSTEMS AND SYSTEM PROGRAMMING

## PRACTICAL – WEEK 1

Name: S Svara Haasini

Roll No: 2520090170

### Task 1: Linux Command Execution Using `fork()`, `exec()` and `wait()`

#### Question:

Develop a C program that demonstrates how a Linux operating system executes a command entered by a user. The program must:

1. Accept a Linux command as input
2. Create a child process using `fork()`
3. Execute the command in the child process using an appropriate `exec()` system call
4. Allow the parent process to wait for the child using `wait()`
5. Display the Process ID (PID) of both parent and child processes

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    char command[100];

    /*
     * Read one Linux command from the user.
     *
     * Examples:
     * ls
     * pwd
     * date
     * whoami
     */
    printf("Enter a Linux command: ");

    if (scanf("%99s", command) != 1)
    {
        printf("Invalid input.\n");
        return 1;
    }

    /*
     * fork() creates a new process.
     */
    pid_t pid = fork();

    if (pid < 0)
    {
        /*
         * A negative value means the child
         * process could not be created.
         */
    }
}
```

```

        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        /*
         * This block runs inside the child process.
         */
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command: %s\n\n", command);

        /*
         * Print the messages before exec replaces
         * the child program.
         */
        fflush(stdout);

        /*
         * Replace the child process with the
         * Linux command entered by the user.
         */
        execlp(command, command, (char *)NULL);

        /*
         * execlp() returns only if execution fails.
         */
        perror("Execution failed");
        _exit(EXIT_FAILURE);
    }
    else
    {
        /*
         * This block runs inside the parent process.
         */
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        /*
         * Wait until the child process finishes.
         */
        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}

```

## Compilation and Execution

```
haasini@SSH:~/OSSP_upload$ gcc -Wall -Wextra command_executor.c -o command_executor
haasini@SSH:~/OSSP_upload$ ./command_executor
Enter a Linux command: ls

Parent Process
Parent PID : 1099
Child PID  : 1100

Child Process
Child PID  : 1100
Parent PID : 1099
Executing command: ls

README.md      command_executor  process_states  task2_outputs
Task2_Report.md  command_executor.c  process_states.c

Child process completed.
haasini@SSH:~/OSSP_upload$ nano command_executor.c
```

### Observations:

- The C program successfully creates a child process using `fork()`, which returns 0 in the child and the child's PID in the parent.
- The parent process displays its PID (1899) and the child's PID (1100) before executing `wait()`.
- The child process displays the same PID information (Child PID: 1100, Parent PID: 1899) and then executes the user-entered command (`ls`) using `execvp()`.
- The `ls` command output is displayed showing the directory contents, confirming successful command execution through `execvp()`.
- The parent process remains blocked in `wait()` until the child completes execution, ensuring proper process synchronization.

## Task 2: Investigation of Hardware Resources and Operating System Services

### Question:

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

System Information (uname, lscpu, ps, top)

```
haasini@SSH: ~/OSSP_upload$ uname -a
Linux SSH 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
haasini@SSH: ~/OSSP_upload$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                48 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                       16
On-line CPU(s) list:         0-15
Vendor ID:                   AuthenticAMD
Model name:                   AMD Ryzen 7 5825U with Radeon Graphics
CPU family:                   25
Model:                       80
Thread(s) per core:          2
Core(s) per socket:          8
Socket(s):                    1
Stepping:                     0
BogoMIPS:                     3992.40
Flags:                        fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse3
                                clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtsc
                                lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpui
                                extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2
                                movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cmp_legacy s
                                m cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoext perfctr_
                                ore ssbd ibrs ibpb stibp vmmcall fsgsbase bmi1 avx2 smep bmi2 erms inv
                                cid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsa
                                es clzero xsaveerptr arat npt nrip_save tsc_scale vmcb_clean flushbyas
                                d decodeassists pausefilter pfthreshold v_vmsave_vmload umip vaes vpc
                                lldq rdpid fsrm
Virtualization features:
Virtualization:               AMD-V
Hypervisor vendor:            Microsoft
Virtualization type:          full
Caches (sum of all):
L1d:                          256 KiB (8 instances)
L1i:                          256 KiB (8 instances)
L2:                            4 MiB (8 instances)
L3:                           16 MiB (1 instance)
NUMA:
NUMA node(s):                  1
NUMA node0 CPU(s):            0-15
Vulnerabilities:
Gather data sampling:          Not affected
Ghostwrite:                    Not affected
```

```
Core(s) per socket:      8
Socket(s):              1
NUMA node0 CPU(s):      0-15
haasini@SSH:~/OSSP_upload$ ps
  PID TTY          TIME CMD
   344 pts/0    00:00:00 bash
  1243 pts/0    00:00:00 ps
haasini@SSH:~/OSSP_upload$ top
top - 04:16:03 up 1:05, 1 user, load average: 0.03, 0.04, 0.00
Tasks: 22 total,  1 running, 21 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 6849.7 total, 6228.3 free,  568.1 used,  201.2 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,   0.0 used,  6281.6 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   21628 13264 9796 S   0.0   0.2   0:01.90 systemd
    2 root        20   0   3180   2204 2072 S   0.0   0.0   0:00.03 init-systemd(Ub
    7 root        20   0   3196   2132 2008 S   0.0   0.0   0:00.00 init
   59 root        19  -1  50444 16380 15308 S   0.0   0.2   0:00.75 systemd-journal
  107 root        20   0   25284 6628   5172 S   0.0   0.1   0:02.79 systemd-udev
  131 systemd+    20   0   21344 13212 11088 S   0.0   0.2   0:00.29 systemd-resolve
  141 systemd+    20   0   91036 7948   6980 S   0.0   0.1   0:00.36 systemd-timesyn
  193 root        20   0   4244   2684 2432 S   0.0   0.0   0:00.04 cron
  194 message+    20   0   9540   5424 4728 S   0.0   0.1   0:00.37 dbus-daemon
  207 root        20   0   17980 8888   7848 S   0.0   0.1   0:00.29 systemd-logind
  209 root        20   0 1756104 11372 8820 S   0.0   0.2   0:00.48 wsl-pro-service
  219 syslog      20   0  222516 5920   4536 S   0.0   0.1   0:00.24 rsyslogd
  226 root        20   0   3124   1972 1836 S   0.0   0.0   0:00.01 agetty
  240 root        20   0 107016 23136 13680 S   0.0   0.3   0:00.37 unattended-upgr
  342 root        20   0   3184   1108   980 S   0.0   0.0   0:00.00 SessionLeader
  343 root        20   0   3300   1008 1100 S   0.0   0.0   0:00.00 bash (awu)
```

**Observations:**

- The uname command displays the Linux kernel version and system architecture (x86\_64), indicating the OS is running on a 64-bit x86 processor.
- The lscpu output shows 16 CPUs (on-line list 0-15) with an AMD Ryzen 7 5825U processor, 8 cores per socket, and 2 threads per core, demonstrating CPU abstraction through logical CPU presentation.
- The system uses Little Endian byte order, which is standard for x86\_64 architecture and affects how the OS organizes memory and I/O operations.
- The ps command shows multiple processes running simultaneously (bash, ps itself, etc.), each with its own PID and TTY, showing how the OS abstracts individual process management.
- The top command output indicates 22 total tasks with 1 running and 21 sleeping, showing memory usage (6849.7 MB total, 6234.6 MB free), demonstrating OS memory abstraction and resource allocation.
- Process states (S = sleeping, R = running) visible in both ps and top outputs show how the OS manages process scheduling and transitions between states without application awareness.